

Logistic Regression (contd.)

MAP formulation

— use a prior on $\vec{\theta}$

$$p(\vec{\theta}) = \mathcal{N}(\vec{0}, \lambda^{-1} \bar{I}_d) \propto \exp(-\lambda \vec{\theta}^T \vec{\theta})$$

map objective: MLE objective + $\log p(\theta)$

$$\begin{aligned} J_{\text{map}}(\theta) &= J_{\text{MLE}}(\theta) + \log p(\theta) \\ &= \sum_{i=1}^n \left[\log(1 + e^{z^{(i)}}) - y^{(i)} z^{(i)} \right] - \lambda \vec{\theta}^T \vec{\theta} \end{aligned}$$

$$J_{\text{map}}(\theta) = - \left[\sum_{i=1}^n (y^{(i)} \theta^T x^{(i)} - \log(1 + \exp(\theta^T x^{(i)}))) + \lambda \vec{\theta}^T \vec{\theta} \right]$$

Regularized Logistic Regression

- $J(\boldsymbol{\theta}) = -\sum_{i=1}^n y^{(i)} \log h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}))$
- We can regularize the logistic regression as

$$\begin{aligned} J_{reg}(\boldsymbol{\theta}) &= J(\boldsymbol{\theta}) + \lambda \sum_{j=1}^d \theta_j^2 \\ &= J(\boldsymbol{\theta}) + \lambda \|\boldsymbol{\theta}\|_2^2 \end{aligned}$$

$$\begin{aligned} J_{reg}(\boldsymbol{\theta}) &= -\sum_{i=1}^n y^{(i)} \log h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})) + \lambda \sum_{j=1}^d \theta_j^2 \end{aligned}$$

Gradient Descent for Logistic Regression

- Objective is to find $\min_{\theta} J(\theta)$

- Initialize θ
- Repeat until convergence

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

simultaneous update
for $j = 0 \dots d$

$$\frac{\partial}{\partial \theta_j} J(\theta) = - \sum_{i=1}^n (y^{(i)} - h_{\theta}(x^{(i)})) x^{(i)}$$

Gradient Descent for Logistic Regression

- Objective is to find $\min_{\theta} J(\theta)$

$$\theta_0 \leftarrow \theta_0 - \alpha \sum_{i=1}^n \left(h_{\theta} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right)$$

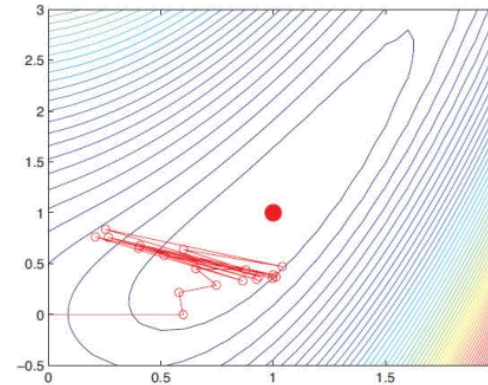
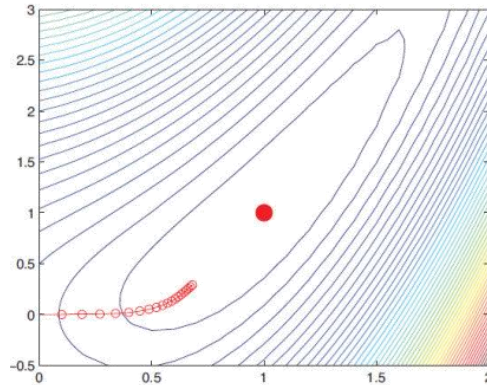
← unregularized

$$\theta_j \leftarrow \theta_j - \alpha \left[\sum_{i=1}^n \left(h_{\theta} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right) x_j^{(i)} - \frac{\lambda}{n} \theta_j \right]$$

← Regularized

More on Gradient Descent

- GD can converge slowly and is also sensitive to the step size



- Choose optimal step size by line-search technique
- Add a momentum term to the updates
- Use second order methods (e.g., Newton's method) to utilize the curvature of the loss function

Gradient Descent with Momentum



$$v_0 = 0$$

$$v_t = -\eta \sum_{k=0}^{t-1} \beta^k \nabla J(\theta_{t-1-k})$$

Exponentially weighted moving average of past gradients.

$$\text{velocity: } v_{t+1} := \beta v_t - \eta \nabla J(\theta_t)$$

$$\theta_{t+1} := \theta_t + v_{t+1}$$

$\beta \in (0, 1)$: damping coefficient or Friction (Typically '0.9')

v : velocity stores exponentially decaying past gradients

η : Time-step scaling / learning rate

Gradient Descent with Momentum

① if gradients point roughly in the same direction
— we double reinforces each other
— speed increases

② If gradient oscillates — +ve and -ve directions cancel
— oscillation is damped.

$$v_{t+1} = \beta v_t - \eta \nabla J(\theta_t)$$

$$\theta_{t+1} = \theta_t + v_{t+1}$$

$\beta = 0 \rightarrow$ No momentum $\rightarrow$ No inertia

$\beta \approx 1 \rightarrow$ No friction / very little fric.
Oscillation explodes $\rightarrow$ instability.

$\beta = 0.9 \rightarrow 90\% \rightarrow$ stable convergence